oad this page to apply your updated settings on this site Reload

37. Ball Collision

There are *n* balls placed on a 1-dimensional axis with each of them moving with the same non-zero speed. *direction[i]* represents the direction in which the i^{th} ball is moving, with -1 meaning that it is moving to the left and 1 meaning it is moving to the right. The strength of the i^{th} ball is described by *strength[i]*. If 2 balls collide, the one with the higher strength destroys the smaller one. If both have the same strength, both are destroyed. Return a list of the indices of the balls that remain after all the collisions have occurred, in ascending order.

Note that the arrays *direction* and *strength* are 0-indexed.

Example

direction = [1, -1]
strength = [2, 1]

The number of balls *n = 2*. The balls are listed in order of occurrence from left to right, so ball 0 is somewhere to the left of ball 1. Ball 0 is moving to the right and ball 1 is moving to the left. The balls will collide at some point and the ball with the higher strength, ball 0, remains. Return [0] as the answer.

Language

Autocomp

```
1  > #in
9    [
10   /*
11    * (
12    *
13    * T
14    * T
15    *
16    *
17    */
18
19   vect
     vect
20
21   }
22
23 > int
```

Test

°F
ostly cloudy

Search

remains. Return [0] as the answer.

Function Description

Complete the function *findRemainingBalls* in the editor below.

findRemainingBalls has the following parameter(s):

int direction[n]: the directions of the particles in order of their starting relative positions

int strength[n]: the strengths of the particles

Returns

int[]: an integer array that contains the indices of the remaining balls in ascending order

Constraints

- $1 \leq n \leq 10^5$
- direction[i] is either 1 or -1
- $1 \leq$ strength[i] $\leq 10^9$

▶ **Input Format For Custom Testing**

Lang

Au

1
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23 >

Tes

ft

▶ Input Format For Custom Testing

▼ Sample Case 0

Sample Input For Custom Testing

```
STDIN       FUNCTION
-----       --------
3       →   the number of balls n = 3
1       →   direction = [1, -1, 1]
-1
1
3       →   the number of balls n = 3
5       →   strength = [5, 3, 1]
3
1
```

Sample Output

```
0
2
```

Explanation

The first and the third balls are moving right and the second ball is moving left. The first and the second balls will collide at some point and the ball with higher strength, ball 0, remains. The third

hackerrank.com/test/g05rfgkbm2l/questions/61dm4chp928

oad this page to apply your updated settings on this site Reload

Sample Input For Custom Testing

```
STDIN       FUNCTION
-----       --------
3       →   the number of balls n = 3
1       →   direction = [1, -1, 1]
-1
1
3       →   the number of balls n = 3
5       →   strength = [5, 3, 1]
3
1
```

Sample Output

```
0
2
```

Explanation

The first and the third balls are moving right and the second ball is moving left. The first and the second balls will collide at some point and the ball with higher strength, ball 0, remains. The third ball does not collide with any ball.

▶ **Sample Case 1**

Language

Autocor

```
1  > #i
9    [
10   /*
11    *
12    *
13    *
14    *
15    *
16    *
17    *
18
19   ve
20
21   }
22
23 > int
```

Test

37°F
Mostly cloudy

Search

Language C++20 Environment

Autocomplete Ready

```
#include <bits/stdc++.h>...

/*
 * Complete the 'findRemainingBalls' function below.
 *
 * The function is expected to return an INTEGER_ARRAY.
 * The function accepts following parameters:
 *  1. INTEGER_ARRAY direction
 *  2. INTEGER_ARRAY strength
 */

vector<int> findRemainingBalls(vector<int> direction,
vector<int> strength) {

}

int main()...
```

Test Custom Run Code Run Tests Submit

18:30
31-07-2024

38. Dominoes Painting

A valid domino arrangement is given which is represented using two strings in the array *domino*. All dominoes are unique (of size 1 x 2 or 2 x 1) and each domino is represented by an English character (lowercase or uppercase).

Each domino can be colored using any one of the colors Red, Green, or Blue (RGB). Find the number of distinct ways to color the dominoes such that no two adjacent dominoes have the same color. Since the answer can be large, return the answer modulo (10^9+7).

Note: Both halves of a domino are one color.

Example

Consider *domino = ["baa", "bcc"].*

This is arranged as:

$$\begin{bmatrix} b & a & a \\ b & c & c \end{bmatrix}$$

The valid coloring of dominoes are: [[RGG, RBB], [RBB, RGG], [GRR, GBB], [GBB, GRR], [BGG, BRR], [BRR, BGG]]. Thus, the answer is 6.

Function Description

Complete the function *countDistinctColorings* in the editor below.

countDistinctColorings has the following parameter:

string domino[2]: two strings of equal length that denote the arrangement of dominoes

Returns:

int: the total number of distinct valid colorings, taking modulo (10^9+7)

Constraints

- $1 \leq (|domino[0]| = |domino[1]|) \leq 52$
- All *domino[i][j]* belongs to ('a' - 'z') or ('A' - 'Z')
- The size of the array is 2 and both strings are of equal length.

▶ **Input Format For Custom Testing**

▼ **Sample Case 0**

Sample Input For Custom Testing

STDIN FUNCTION

▶ **Input Format For Custom Testing**

▼ **Sample Case 0**

Sample Input For Custom Testing

```
STDIN                FUNCTION
-----                --------
2                →   number of strings in array domino = 2
xy               →   domino[] = ["xy", "xy"]
xy
```

Sample Output

```
6
```

Explanation

The arrangement can be represented as:

$$\begin{bmatrix} x & y \\ x & y \end{bmatrix}$$

All valid colorings are: [[RG, RG], [RB, RB], [GR, GR], [GB, GB], [BR, BR], [BG, BG]].

▶ **Sample Case 1**

guage C++20 Environment Autocomplete Ready

```
> #include <bits/stdc++.h> ···

/*
 * Complete the 'countDistinctColorings' function below.
 *
 * The function is expected to return an INTEGER.
 * The function accepts STRING_ARRAY domino as parameter.
 */

int countDistinctColorings(vector<string> domino) {

}

> int main() ···
```

Test Results Custom Input Run Code Run Tes

Search

DELL

```
cin>>n;
string s1, s2;
cin>>s1>>s2;
int prev; // 0 -> vertical, 1 -> horizontal
int i = 0;
long long ans;

if(s1[0] == s2[0]){
    prev = 0;
    ans = 3;
    i = 1;
} else {
    prev = 1;
    ans = 6;
    i = 2;
}

for(i ; i<n ;){

    if(s1[i] == s2[i]){ // v
        if(prev == 0)
            ans = (ans*2)%mod;
        prev = 0;
        i += 1;
    }
    else{
        if(prev == 0)
            ans = (ans*2)%mod;
        else
            ans = (ans*3)%mod;

        prev = 1;
        i += 2;
    }
}

cout<<ans;
```